Degrade → Restore → Detect: Notebook Explainers

Oct 28, 2026 · @Shourja Banerjee

A study guide to the six notebooks in this repository, one tab per notebook. Each tab gives the notebook's purpose, explains every function, says what the outputs mean, and ends with self-check questions. The notebooks themselves carry short explanations; this guide goes into more depth.

How to use these explainers

Read one tab with its notebook open beside it. Each tab follows the notebook's own section order and uses its function names, so you can match every paragraph to a cell.

Each tab has four parts:

1. **Purpose:** the question the notebook answers, and what it hands to later notebooks.
2. **Section by section:** each function's inputs and outputs, what it does, and why it was written that way.
3. **What the output tells you:** the real numbers from the saved run, and how to read them.
4. **Check yourself:** questions to test your understanding. If one takes more than two sentences to answer, reread that part.

A good way to study: before reading a section, try explaining each cell from memory in one line on *what* it does and one on *why*, then check against the guide. For example, for `noise_rng`:

> Makes a separate random generator for each (image, degradation, severity), so every filter is tested on exactly the same noise. That makes the comparison paired, so differences come from the filter, not from luck.

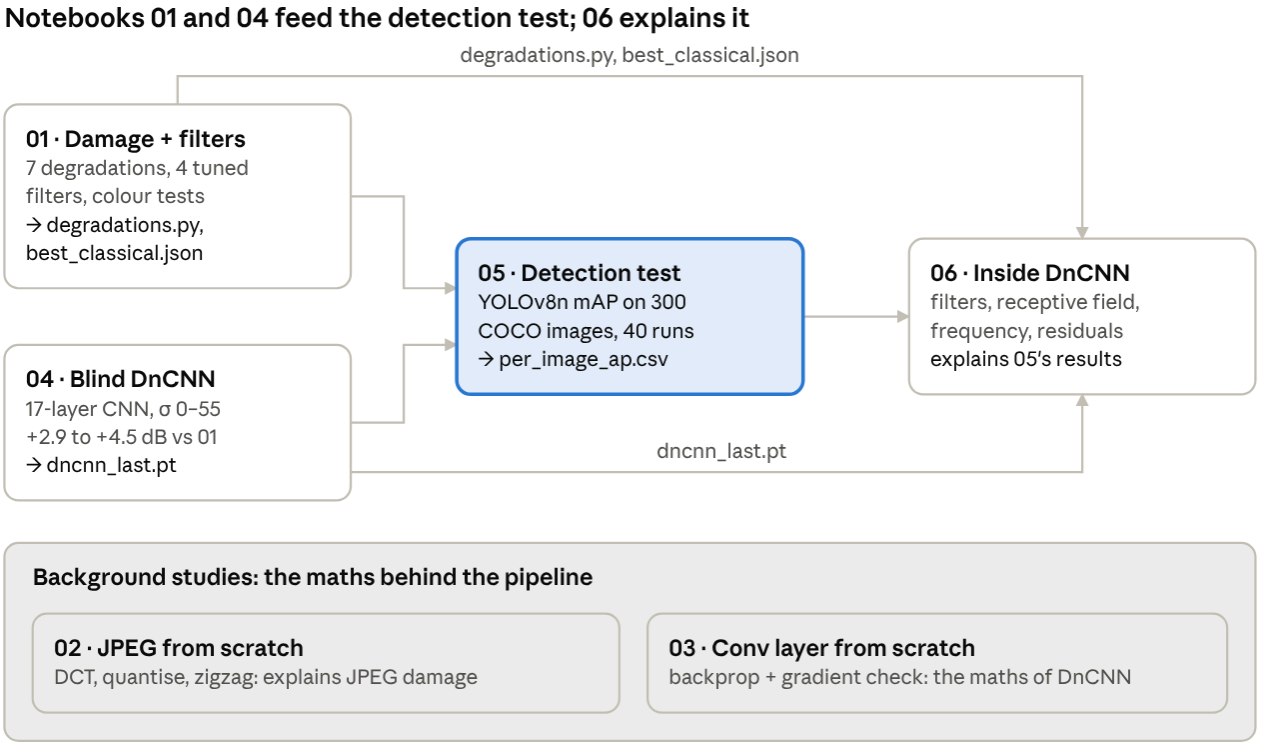
Written from the notebooks in `notebooks/` and the outputs saved in them; the analysis CSVs are in `results/`.

Conventions every notebook shares

These rules are why the notebooks can pass data to each other and why comparisons between methods are fair.

Rule	What it means	Why
Image format	`float32`, **RGB**, shape `(H, W, 3)`, values in **[0, 1]**	One format everywhere means no conversion bugs between notebooks. Notebook 02 is the exception: it works in 0–255 because real JPEG does.
OpenCV is BGR uint8	`cv2.imread` returns BGR in 0–255, so `load_rgb` flips it with `[:, :, ::-1]` and divides by 255	Forgetting this swaps red and blue, or breaks maths through uint8 overflow (250 + 10 wraps to 4).
Degradation signature	`f(img, severity, rng) -> img`	Every degradation can be called the same way through `degrade(img, name, severity, rng)`.
Clip once	Functions don't clip; `degrade()` clips to [0, 1] at the end	Clipping inside each function would hide mistakes and change the noise statistics twice.
All randomness via `rng`	`np.random.default_rng(...)`, never `np.random.randn`	Results are reproducible, and you control exactly which noise each image gets.
Paired noise	The generator is seeded from `[SEED, image index, degradation index, severity index]`	Every restorer sees the identical noisy image, so a difference between methods is caused by the method, not by luck.
Tuning vs test split	First 4 BSD500 val images tune filter settings; the other 96 are only for reporting	Tuning on the images you report would inflate the classical scores.
Train vs test	DnCNN trains on BSD500 `train` + `test` (400 images), evaluates on `val`	It is never scored on images it learned from (notebook 04 checks this by name and by pixel hash).

How the notebooks fit together



data flow between the six notebooks

Notebooks 01 and 04 build the two kinds of repair, 05 tests whether they help detection, and 06 opens up DnCNN to explain why. On Kaggle, files move between notebooks through *Add Input → Notebooks*, which pins a specific committed version, so re-add the input after you re-commit a notebook. Notebooks 02 and 03 pass no files on: they build the maths behind JPEG damage and behind DnCNN from scratch.

01 · Degradations and classical baselines

Purpose

Notebook 01 builds the damage and the classical repairs that the rest of the project uses. It answers: **how well can hand-designed filters undo each kind of damage, and when do they fail?** It hands three things on: `degradations.py` (to 05 and 06), `best_classical.json` (the tuned filter per condition, to 05 and 06), and the classical PSNR numbers that DnCNN has to beat in 04.

Section 0–1 · Setup and data

- **`load_rgb(path)`**: reads an image with OpenCV, flips BGR → RGB with `[:, :, ::-1]`, converts to float in [0, 1]. Every image enters the project through this.
- **`list_images(folder)`**: finds all image files recursively and sorts them, so the order (and therefore the tuning/test split) is always the same.
- **`show_grid(...)`**: plots images in a grid and optionally saves the figure. A helper, not part of the method.
- **`check(cond, msg)`**: prints `PASS` or `FAIL`. All the test cells use it.
- **Data cell**: loads the 100 BSD500 `val` images. The first 4 become `tune_imgs` and the other 96 `test_imgs`. Output: 481 × 321 images, float32, range [0, 1].
- **Histogram cell**: shows that the R, G and B histograms look alike. In natural images the channels are strongly correlated, which is the basis of the colour-space experiment in section 6.

Section 2 · `degradations.py`

`%%writefile` saves the cell as a Python file, so later notebooks can `from degradations import degrade`. Each function models one physical cause of damage.

Function	Model	Severity knob	Key idea
`add_gaussian_noise`	$y = x + n$, $n \sim N(0, (\sigma/255)^2)$	σ on the 0–255 scale: 5–50	"White" = each value independent; same strength everywhere

Function	Model	Severity knob	Key idea
`add_poisson_noise`	y = Poisson(peak · x) / peak	photon count `peak`: 100 → 5	Variance = x / peak, so the noise depends on the signal; SNR = √(peak · x) is worst in dark areas
`add_salt_pepper`	a fraction of pixels set to 0 or 1	fraction: 0.01–0.2	Per *pixel* (all 3 channels together), like dead sensor pixels
`gaussian_blur`	convolution with a Gaussian kernel	kernel σ in pixels: 0.5–3	`(0, 0)` lets OpenCV size the kernel (≈ 6σ); reflect borders so edges don't darken
`jpeg_compress`	encode + decode with OpenCV	quality: 90 → 10	The one place that leaves the convention: converts to uint8 BGR for OpenCV, then back
`to_grayscale`	luma 0.299R + 0.587G + 0.114B, or the plain average	method	Returns 3 equal channels so shapes stay `(H, W, 3)`
`color_cast`	gains (1 + s, 1, 1 − s) on R, G, B	strength: 0.1–0.5	Wrong white balance; s = 0 is exactly the identity
`low_light`	gain · x^γ, then Poisson shot noise, then Gaussian read noise	level: 0.2–0.9	Darker images also get noisier, as in a real camera; level 0 skips the noise so it is an exact identity

- **`DEGRADATIONS`**: a dictionary mapping each name to `(function, list of severities from mild to severe)`. Loops elsewhere iterate over it.
- **`degrade(img, name, severity, rng)`**: the single entry point. It looks up the function, runs it, and clips to [0, 1] **once**.

Tests (`run_degradation_tests`) check statistics, not just shapes, mostly on a flat grey image so clipping can't distort the numbers. In the saved run: Gaussian std 25.01 (want 25); Poisson variance 0.0247 (want 0.0250); 9.9% of pixels changed by salt & pepper (want 10%); JPEG error falls as quality rises; pure red gives luma 0.299 and average 0.333; low-light brightness falls steadily with level. All tests pass.

Section 3 · Metrics

- **`psnr(ref, test)`**: computes MSE in `float64`, then $10 \cdot \log_{10}(1 / \text{MSE})$ dB, and returns infinity when the images are identical. +6 dB means the RMS error halved.
- **`ssim(ref, test)`**: scikit-image's SSIM, which compares local brightness, contrast and structure. It is closer to human judgement than PSNR.
- **Tests**: a constant +0.1 error must give exactly 20 dB, and a random ±0.1 error must also give 20 dB. If the second test failed, you would be squaring the *mean* error instead of each error.

Section 4 · Classical filters

Each wrapper takes and returns float RGB in [0, 1]. `to_u8` / `from_u8` convert for the OpenCV functions that need uint8.

Filter	How it works	Best for	Fails when
`f_median(k)`	middle value of each k × k window	salt & pepper: one wrong pixel can't move a median	dense Gaussian noise; thin lines
`f_gaussian(sigma)`	weighted local average	mild Gaussian noise	edges matter: it blurs everything equally
`f_bilateral(sigma_color, sigma_space)`	averages only neighbours that are close in position **and** in colour	noise on images with strong edges	heavy noise, because noise itself starts to look like edges
`f_nlmeans(h)`	averages pixels whose surrounding 7 × 7 *patches* look alike, anywhere in a 21 × 21 area	Gaussian noise, repeating texture	very heavy noise; slow (277 ms per image vs 1–5 ms for the others)

`PARAM_GRID` lists the parameter values tried for each filter during tuning.

Section 5 · Experiment 1: tuning and severity sweep

- **`noise_rng(img_idx, deg_name, sev_idx)`**: makes a generator seeded from those indices. The same image and condition always get the same noise, whichever filter is being tested. This is what makes the comparison *paired*.
- **Tuning cell**: for every degradation, severity and filter, tries every parameter setting on the 4 tuning images and keeps the one with the best mean PSNR. If no filter beats doing nothing, it records `"none"` (true for all blur levels). It also warns when the best value sits at the edge of the grid, since the real optimum may lie beyond it. Saves `best_classical.json`.
- **Sweep cell**: applies each tuned filter to the 96 test images and stores PSNR and SSIM per image in `baselines.csv`.
- **`plot_curves`**: PSNR and SSIM against severity, one line per filter (Fig. 2 of the report).
- **`win_rate`**: on what share of images the best filter beats doing nothing. A mean can hide a few large wins, so this shows whether a win is consistent.
- **`compare_figure`**: one zoomed crop per filter, for the qualitative figure.
- **Blind NL-means (optional)**: `estimate_sigma_haar` estimates the noise level from the noisy image itself. Noise lives in the finest diagonal Haar band, and median(|HH|) / 0.6745 turns a median into a standard deviation (the median is robust to the few real edges in that band). The cell fits $h \approx 0.60\sigma$ from the tuned settings, then runs NL-means without being told σ.

Fairness point: the tuned filters were told the true severity, an "oracle" advantage DnCNN doesn't get. The blind NL-means test shows the advantage is small: not knowing σ costs only 0.04–0.31 dB for $\sigma \geqslant 15$. So a DnCNN win is a real win.

Section 6 · Experiment 2: colour space

- **Noise check (6.1)**: each YCrCb channel is a weighted sum $w \cdot (R, G, B)$, so its noise std is $\sigma\|w\|$. Theory and measurement agree (Y 0.669 / 0.667, Cr 0.657 / 0.658, Cb 0.623 / 0.625). So the noise is about equal in all three channels, but the image's detail is not: detail-to-noise is 1.27 in Y, 0.06 in Cr and 0.10 in Cb. The colour channels are almost pure noise.
- **`denoise_in_space(img, space, strengths)`**: converts to RGB, YCrCb or Lab, blurs each channel with its own strength, converts back. Only the colour space changes between configurations, never the filter.
- **Sanity check**: YCrCb with equal strengths must equal RGB exactly (max difference 7.9e-05), because a linear colour transform and a linear blur commute. This is a free bug check.

- **Result**: blurring luma gently and chroma heavily, YCrCb (0.7, 2, 2), beats plain RGB by +2.07, +1.51 and +0.68 dB at σ = 15, 25, 50, on 99–100% of images. The reverse setting is much worse. JPEG's chroma subsampling (notebook 02) and DnCNN's colour-opponent filters (notebook 06) rely on the same fact.

Section 7 · Experiment 3: grayscale and contrast

- **Luma vs average (7.1)**: pure R, G and B have the same average (0.333) but luma 0.299, 0.587 and 0.114. Any conversion from 3 numbers to 1 merges some colours. The cell builds one colour pair that each method can't tell apart.
- **Contrast (7.2)**: `hist_eq_rgb`, `hist_eq_L`, `clahe_rgb` and `clahe_L` brighten low-light images. `_per_channel` applies a function to R, G, B separately; `_on_L` applies it to Lab's lightness only, leaving colour alone. `color_shift` measures colour change as the mean distance in Lab's (a, b) plane.
- **What the output shows**: equalising each RGB channel separately shifts colours badly (colour shift 38.6 vs 14.9 for HE on L). PSNR stays low (10–14 dB) even when the image looks far better, because brightening doesn't restore the true brightness. That's another case of a metric disagreeing with how an image looks.

Section 7.5 · Fixes that match the damage

- **`gray_world(img)`**: assumes the scene averages to grey and scales each channel to the overall mean. At cast 0.3 it raises PSNR from 20.0 to 25.1 dB, where denoisers did nothing. It fails on a scene that really is one colour: on clean image #15 it drops PSNR to 15.7 dB.
- **`gaussian_psf`** and **`wiener_deblur(img, sigma, K)`**: inverts a known blur in the frequency domain, $X = H^* / (|H|^2 + K) \cdot Y$. K stops division by near-zero frequencies, which would amplify noise. The image is reflect-padded first because the FFT assumes it wraps around. At blur 1.0 with no noise it lifts PSNR from 28.0 to 37.5 dB. With a little noise the best K grows from 1e-4 to 1e-2.
- **The lesson**: denoisers can't fix blur or colour cast. The repair has to match the damage model. Notebook 05 shows the same thing for detection.

Section 8 · Wrap-up

Builds `best_filter_table.md` (best filter and gain per condition) and checks that every file later notebooks need exists. Key numbers: the best classical PSNR on Gaussian noise is 30.90 / 27.56 / 23.20 dB at σ = 15 / 25 / 50, mostly from the bilateral filter.

Check yourself

- [ ] Why is noise generated in float, and why does `degrade()` clip only once?
- [ ] Why is Poisson noise stronger in bright regions in absolute terms, but weaker relative to the signal there?
- [ ] Why does the median filter beat the Gaussian filter on salt & pepper noise?
- [ ] What does `noise_rng` guarantee, and what would go wrong without it?
- [ ] Why does tuning on 4 separate images matter, and what is the "oracle" advantage?
- [ ] Why must YCrCb (1, 1, 1) equal RGB (1, 1, 1) exactly?
- [ ] Why can chroma be blurred much more than luma?
- [ ] Why doesn't any denoiser help on blur, and what does Wiener's K do?

02 · JPEG from scratch

Purpose

Notebook 02 rebuilds the lossy core of baseline JPEG in NumPy, to work through compression mathematically. It answers: **where does JPEG throw information away, and why does it cost so little visible quality?** The codec matches OpenCV's to within 0.1 dB at the same quality up to q = 90. It explains the JPEG damage used in notebooks 01 and 05.

The pipeline: RGB → YCbCr → 4:2:0 chroma subsampling → 8 × 8 blocks → level shift → DCT → quantise → zigzag → entropy estimate, then the exact inverse to decode.

Only quantisation loses information. Every other step can be undone exactly (chroma subsampling also loses some, but only colour detail). Keep this in mind for every section below.

Setup

- **`load_rgb`**: here images stay in 0–255, because JPEG works on 8-bit values.
- **`crop16`**: crops height and width to multiples of 16. 4:2:0 halves the chroma, and then it must still split into 8 × 8 blocks, so 16 is the unit.
- BSD500 images are already stored as high-quality JPEGs, so "original" means "as distributed".

Section 1 · Colour transform and subsampling

- **`M_YCC`**, **`OFFSET`**: the JFIF full-range BT.601 matrix. Y is brightness; Cb and Cr are blue-difference and red-difference, offset by 128 so they fit in 0–255.
- **`rgb_to_ycbcr`** / **`ycbcr_to_rgb`**: a 3 × 3 matrix multiply per pixel. The inverse exists because det(M) = 0.2363 ≠ 0, so this step loses nothing (the assert checks the round trip).
- **`subsample_420(c)`**: averages each 2 × 2 block, halving the resolution in both directions. The `reshape(H/2, 2, W/2, 2).mean(axis=(1, 3))` trick does it without loops.
- **`upsample_420(c, method)`**: back to full size, by repeating values (`nearest`) or interpolating (`bilinear`, close to libjpeg's "fancy upsampling"). This choice turns out to matter a lot at high quality (Experiment 4).

Why chroma and not luma: the eye resolves fine detail in brightness much better than in colour, and natural images carry most of their detail in Y (the detail-to-noise numbers from notebook 01, section 6).

Section 2 · The DCT as a matrix

- **`dct_matrix(N)`**: builds the orthonormal DCT-II matrix:

```
C_{k,n} = \alpha_k \cos\left(\frac{\pi (2n+1) k}{2N}\right), \quad \alpha_0 = \sqrt{1/N},\ \alpha_{k>0} = \sqrt{2/N}
```

- **`dct2(block, C)`** = C B C^T, and **`idct2(coeffs, C)`** = C^T F C. Because C is orthonormal (C C^T = I), the inverse is just the transpose. It also preserves energy (Parseval): the sum of squared pixels equals the sum of squared coefficients.
- **Asserts**: C C^T = I; `dct2` matches `scipy.fft.dctn`; `idct2` undoes `dct2`; energy is preserved. SciPy is used only to check, never inside the codec.
- **Basis plot**: the 64 basis images are outer products of rows of C. Top-left is flat (DC); moving right or down adds horizontal or vertical stripes of rising frequency.

Section 3 · Quantisation

- **`Q_LUMA`**, **`Q_CHROMA`**: the standard tables from the JPEG spec (ITU-T T.81, Annex K). Step sizes grow toward high frequencies, and the chroma table is coarser overall.
- **`scale_qtable(Q, quality)`**: libjpeg's quality scaling. s = 5000 / q below 50 and 200 − 2q from 50 up, then Q_q = clip(⌊(Q · s + 50) / 100⌋, 1, 255). At q = 50 the table is unchanged (assert). Luma DC step: 80 at q = 10, 16 at q = 50, 3 at q = 90.
- **`quantise(F, Qq)`** = round(F / Qq), **`dequantise`** = Fq · Qq. The rounding is the **one** place information is lost: every coefficient becomes the nearest multiple of its step.

Section 4 · Zigzag and entropy

- **`zigzag_indices(N)`**: orders the 64 positions by anti-diagonal (i + j), alternating direction, so low frequencies come first and the many zeros cluster at the end. The assert checks the first six positions.
- **`entropy_bits(symbols)`**: Shannon entropy × number of symbols, the ideal code length. It stands in for Huffman coding, which is not implemented, so the bit counts are an *estimate*.

Section 5 · Encoder and decoder

- **`blockify` / `unblockify`**: (H, W) ↔ (H/8, W/8, 8, 8) with reshape and `swapaxes`, no loops.
- **`encode_channel(ch, Qq)`**: subtract 128 (the level shift centres values around 0, so the DC coefficient is small), DCT, quantise. **`decode_channel`**: the reverse.
- **`size_cat(v)`**: JPEG's size category, the number of bits needed for |v|.
- **`channel_bits_simple`**: a naive estimate, the entropy of every coefficient value.
- **`channel_bits`**: closer to real JPEG. DC is coded as the *difference* from the previous block (neighbouring blocks have similar averages). AC values become (zero-run, size) symbols plus amplitude bits, with ZRL for 16 zeros and EOB for "the rest of the block is zero". The run-length version is 4–28% smaller than the naive one (the most at low quality, where zeros dominate), which shows why run-length coding exists.
- **`my_jpeg(img, quality, ...)`**: the whole codec. It returns the decoded image (rounded and clipped to 8 bits, as a real decoder outputs) and the estimated bits.
- **`opencv_jpeg`**: the reference, with the real file size. **`psnr255`**: PSNR on the 0–255 scale.

At quality 50: 1.41 bits per pixel, 27.90 dB.

Section 6 · Experiments

1 · Rate–distortion. Its PSNR matches OpenCV's at every quality up to 90 (e.g. 27.90 vs 27.90 dB at q = 50, 34.87 vs 34.82 at q = 90). Across 20 images the mean gap is −0.11 dB. That mean is pulled down by q = 95 (48.85 vs 49.78 dB on the working image), where the small differences in chroma upsampling and rounding are no longer hidden by quantisation error. The estimated size is 0.86× the real file (0.69× at q = 10), because ideal entropy beats real Huffman tables and file headers are not counted. So at a matched bit rate the curve looks +0.88 dB better, but that is an artefact of the estimate, not a better codec.

2 · Energy compaction. DC alone holds 84.6% of the luma energy, and the first 6 of 64 coefficients 92.5%. At q = 50, 73.8% of luma coefficients quantise to exactly 0. This is why JPEG works: the DCT concentrates energy in a few coefficients, so coarsely quantising the rest costs little.

3 · Artifacts at q = 10. Pixel steps across 8 × 8 block boundaries are 2.1× larger than inside blocks (19.13 vs 8.92), while the original has no block structure (15.48 vs 15.84). *Blocking* happens because each block is coded independently. *Ringing* next to edges happens because removing high frequencies truncates the edge's Fourier series (the Gibbs phenomenon). These false edges are what notebook 05's bilateral filter smooths away to help the detector.

4 · Ablation. 4:2:0 saves 8–10% of bits and costs 0.17–0.65 dB with bilinear upsampling. With nearest upsampling the cost at q = 95 jumps to 4.91 dB: at high quality almost all the remaining error comes from how chroma is upsampled.

Check yourself

- [ ] Which single step loses information, and why can every other step be inverted?
- [ ] Why is the inverse DCT just a transpose? What does Parseval's theorem say here?
- [ ] Why are quantisation steps larger for high frequencies and for chroma?
- [ ] Why subtract 128 before the DCT?
- [ ] Why does zigzag order help compression?
- [ ] Why is the bit count an estimate, and why does that make this codec's curve look better than OpenCV's at the same bit rate?
- [ ] Explain blocking and ringing from the maths.
- [ ] Why does the upsampling method matter most at q = 95?

03 · A convolution layer from scratch

Purpose

Notebook 03 works out what a convolution layer computes and how gradients flow back through it, with code instead of quoted formulas. It answers: **is the derivation of the conv layer's forward and backward pass correct?** The proof is a numerical gradient check that agrees with the analytic gradients to about 1e-10. It is the maths under DnCNN (notebook 04), and section 6 shows the layer can learn to denoise.

Shapes follow PyTorch: input `x` is (N, C_in, H, W), weights `w` are (C_out, C_in, k, k), bias `b` is (C_out,).

Setup

Imports SciPy's `correlate2d` and `convolve2d` as independent references, and PyTorch if it's installed. In the saved run PyTorch wasn't available, so the PyTorch comparisons were skipped and the SciPy references and numerical gradient check did the verifying. `float64` is used throughout because gradient checks need the precision.

Section 1 · The maths

The forward pass: each output value is a dot product between one kernel and one input patch, plus a bias.

```
y_{n,o,i,j} = b_o + \sum_{c}\sum_{u,v} w_{o,c,u,v}\, \tilde
x_{n,c,\,si+u,\,sj+v}, \qquad H_{out} = \left\lfloor \frac{H + 2p - k}{s}
\right\rfloor + 1
```

This is technically cross-correlation (the kernel isn't flipped). It doesn't matter for learning, because a flipped kernel is just another set of weights.

With g = ∂L/∂y coming from the layer above:

- **Bias**: b_o is added at every position of output channel o, so its gradient is the sum of g over the batch and all positions.
- **Weights**: w_{o,c,u,v} multiplies a particular input value at every position, so its gradient is the sum of g times that input: a correlation of the input with the upstream gradient.
- **Input**: each input pixel feeds every output whose window covers it. For stride 1 this works out to a *full* correlation of g with the kernel **rotated by 180°**, which is the same as a true convolution. The rotation appears because the input sees the window from the other side.

Section 2 · Forward pass

- **`conv2d_forward_naive(x, w, b, stride, pad)`**: zero-pads the input, computes the output size, then loops over output positions (i, j). At each one it takes the patch (N, C, k, k) and uses `np.tensordot` to dot it with all kernels at once, giving (N, O). It returns `y` and a `cache` holding what the backward pass needs.
- **`relu_forward`** / **`relu_backward`**: ReLU is max(x, 0). Its gradient passes `dout` through only where the input was positive.
- **`conv_reference_scipy`**: an independent version built from SciPy's `correlate2d`, summed over input channels, then subsampled for the stride.
- **Test**: three cases with different shapes, strides (1, 2) and paddings (0, 1, 2) all match SciPy to 1e-10.

Section 3 · Backward pass and gradient check

- **`conv2d_backward_naive(dout, cache)`**: the same loop as the forward pass, run backwards. At each output position, the gradient g (N, O) is sent to the kernel (`dw += g · patch`) and to the patch it came from (`dxp += g · w`). It uses `+=` because overlapping windows send gradient to the same input pixel more than once. `db` is just `dout` summed over batch and positions. Finally, the padding is cropped off `dxp` to give `dx`.
- **`numerical_grad(f, arr, eps)`**: central differences. Nudge one element up by ε, then down, and take $(f_+ - f_-) / 2\varepsilon$. Repeat for every element. It's slow but needs no calculus, so it is an independent check.
- **`rel_error(a, b)`**: $\|a - b\| / (\|a\| + \|b\|)$. A norm-based error is used because with stride 2 some input pixels get exactly zero gradient, and an element-wise ratio there would just measure rounding noise.
- **The check**: the loss is L = sum(y · g) for a random fixed g, so $\partial L/\partial y = g$ exactly. Result: relative errors 1.7e-10 to 6.9e-10 for dx, dw and db at strides 1 and 2, pads 0 and 1. The threshold is 1e-7.
- **"The check has teeth"**: deliberately using an unrotated kernel gives a relative error of 0.71. This proves the test would catch a real mistake, not just pass everything.

Section 4 · The rotated kernel, numerically

For stride 1, the cell computes dx three independent ways: the backward pass; a full correlation of g with the 180°-rotated kernel; and a full convolution of g with the unrotated kernel. All three agree. This is the cleanest evidence that section 1's derivation is right.

Section 5 · im2col

- **`im2col(x, k, stride, pad)`**: copies every receptive field into one row of a matrix, shape (N · H_out · W_out, C · k · k). It uses `sliding_window_view`, so there is no Python loop over positions.
- **`conv2d_forward`**: the whole layer is then one matrix multiply, Y = X_col · W^T.
- **`col2im`**: the backward of im2col. It scatter-adds each row back to the input positions it was copied from. It loops k × k times (9) instead of H_out × W_out times.
- **`conv2d_backward`**: `dw` = g^T · X_col, `db` = sum of g, `dx` = col2im(g · W).
- **Results**: matches the naive layer on all cases. It is 2× faster than the "naive" layer, but that layer was already vectorised by `tensordot`. Against true pure-Python loops it is 248× faster.
- **Memory cost**: the im2col matrix is 9.4 MB, 9× the input, because each pixel is copied k × k = 9 times. GPUs accept this because one big matrix multiply runs far faster than many small operations.

Section 6 · Tiny NumPy denoiser

- **What it is**: conv(1→16) → ReLU → conv(16→16) → ReLU → conv(16→1), built only from this layer and trained to predict the **noise** in 32 × 32 grayscale patches at σ = 25 (the same idea as DnCNN).
- **`sample_batch`**: random crops from 50 BSD500 training images. **`he(o, c, k)`**: He initialisation, std $\sqrt{(2 / \text{fan-in})}$, which keeps activation sizes stable through ReLU layers. The last layer starts 10× smaller so the first predictions are close to zero.
- **`forward`** / **`backward`**: chain the layers and their caches, then send gradients back in reverse order.
- **Training loop**: Adam written out by hand, with first and second moments and bias correction. The gradient of mean((pred − noise)²) is 2 · diff / size.
- **Result**: loss fell from 0.00798 to 0.00210 in 400 steps (23 s). Always predicting zero would score σ² = 0.00961. On a held-out crop: 20.65 dB noisy → 25.43 dB denoised (+4.77 dB). The hand-written layer learns.

Check yourself

- [ ] Derive the output-size formula. What is it for H = 7, k = 3, p = 1, s = 2?
- [ ] Why does `dx` involve the kernel rotated by 180°?
- [ ] Why does the backward pass use `+=` for `dxp`?
- [ ] How does central differencing work, and why use float64?

- [ ] Why use a norm-based relative error instead of an element-wise one?
- [ ] What does the "deliberately wrong" test prove?
- [ ] What does im2col trade, and why do GPUs like it anyway?
- [ ] Why predict the noise instead of the clean image?

04 · Training a blind DnCNN

Purpose

Notebook 04 trains the learned restorer: one colour DnCNN that removes Gaussian noise of any strength from 0 to 55 without being told how strong it is ("blind"). It answers: **can a learned denoiser beat the best tuned classical filter, without the oracle advantage?** It does, by +2.9 to +4.5 dB. It hands `dncnn_last.pt` to notebooks 05 and 06.

The key idea is **residual learning**: the network predicts the noise $\hat{v} = f(y)$, and the clean estimate is $y - \hat{v}$. Predicting the noise is easier than predicting the whole image, because noise has a simple structure and the identity map is already nearly right.

Workflow: develop with `QUICK = True` (5 epochs, 25 images per folder), then set it to `False` and commit the 50-epoch run on Kaggle's GPU.

Setup and `Config`

`Config` is a dataclass holding every setting, so the whole run is described in one place and saved into the checkpoint.

Setting	Value	Why
`train_dirs` / `test_dir`	BSD500 `train` + `test` (400 images) / `val` (100)	Test images are never trained on
`patch`, `stride`	40, 20	Small overlapping crops give many training samples
`scales`	1.0, 0.9, 0.8, 0.7	Same textures at several sizes, more variety from few images
`depth`, `features`	17, 64	DnCNN-S's depth; the paper's blind model uses 20. Fewer layers train faster on Kaggle's GPU, and 17 still came within about 0.2 dB of the paper
`sigma_max`	55	Noise σ drawn uniformly from [0, 55] per patch
`batch`, `epochs`, `lr`	128, 50, 1e-3	Adam, learning rate cut 10× at epochs 30 and 42

Seeds are set for torch, NumPy and Python's `random`, so the run is reproducible.

Section 1 · Patch dataset

- **`extract_patches(folder, patch, stride, scales)`**: for each image and scale, resizes and then takes every 40 × 40 window at step 20. `sliding_window_view(...)[::stride, ::stride]` does this with no loop over positions. Patches are kept as **uint8**, 4× smaller than float32. In the saved run: 381,200 patches, 1.83 GB.
- **`augment(p, k)`**: one of the 8 rotations and flips (the "dihedral group"). It's applied on the fly in `__getitem__`, not stored, because storing all 8 versions would make the array 8× bigger for nothing.
- **`check_no_overlap(train_dir, test_dir)`**: compares file names **and** an MD5 hash of the pixels, because the same image could have different names. In the saved run: 0 shared names and 0 identical images for both training folders.
- **`NoisyPatches(Dataset)`**: `__getitem__` picks a patch, augments it, converts to a (3, 40, 40) float tensor, draws one σ ~ U(0, 55), adds noise, clips to [0, 1], and returns `(noisy, noise, clean)`. Fresh noise every time means every epoch sees new training pairs. The target `noise` is computed **after** clipping, so `noisy − noise = clean` exactly, and training matches the test pipeline, where `degrade()` also clips.
- **Sanity checks**: shapes, dtype and `noisy − noise == clean`; noise std over 2,000 items averages 26.2 (just under σ_max / 2 = 27.5 because of clipping), maxing at 53.4.
- **`DataLoader`**: shuffles, 2 worker processes, `pin_memory` for faster GPU copies, `drop_last` so every batch is a full 128.
- **Predict-zero baseline**: a model that always outputs "no noise" scores 0.01371, the average noise power. Training loss must end well below this.

Section 2 · Model

- **`DnCNN(channels, depth, features)`**: layer 1 is Conv 3 × 3 + ReLU. Layers 2 to 16 are Conv 3 × 3 (no bias) + BatchNorm + ReLU. Layer 17 is Conv 3 × 3 back to 3 channels. Padding 1 keeps the output the same size as the input.
- The middle convs have no bias because BatchNorm's own shift would make it redundant.
- `forward` returns the predicted **noise**; the clean image is `y − model(y)`.
- 558,403 parameters.

Section 3 · Training loop

- **Optimiser**: Adam at lr 1e-3, with `MultiStepLR` dividing it by 10 at 60% and 85% of training. Large steps first to make progress, small steps later to settle.
- **Mixed precision**: `torch.autocast` runs most operations in float16 for speed; `GradScaler` multiplies the loss before backprop so small float16 gradients don't round to zero. The loss itself is computed in float32 (`pred.float()`).
- **Loss**: `mse_loss(pred, noise)` , the mean squared error between predicted and true noise. The paper's $(1/2N)\|f(y) - v\|^2$ differs only by a constant factor, which doesn't change the optimum.
- **Order each step**: zero the gradients, backprop through the scaler, step the optimiser, update the scaler.
- **Checkpoint and resume**: the model, optimiser, scheduler, epoch, loss history and config are saved after every epoch. If the run is interrupted, it resumes where it stopped. The QUICK run writes a different file, so it can never be resumed into the real one.
- **Reading the log**: since loss = MSE of the restored image, $10 \cdot \log_{10}(1 / \text{loss})$ is the average training PSNR. The loss fell from 0.00548 (epoch 0) to about 0.00108 by epoch 20, around 29.7 dB, at about 275 s per epoch.

Section 4 · Evaluation

- **`denoise(model, img)`** : `model.eval()` makes BatchNorm use its stored running statistics instead of the batch's. Then it converts (H, W, 3) to (1, 3, H, W), subtracts the predicted noise, clips, and converts back.
- **`noisy_like_nb01(clean, i, sigma)`** : rebuilds exactly the noise notebook 01 used for test image i, so DnCNN and the classical filters are scored on identical noisy images.
- **Test images**: the same 96 as notebook 01 (the first 4 were used for tuning there).

σ	Noisy PSNR	DnCNN PSNR	Best classical (nb 01)	DnCNN gain over classical
15	24.82 dB	33.76 dB	30.90 dB	+2.9 dB
25	20.53 dB	31.05 dB	27.56 dB	+3.5 dB
50	15.00 dB	27.69 dB	23.20 dB	+4.5 dB
65 (outside training)	13.09 dB	26.16 dB	not tuned	—

σ	Noisy PSNR	DnCNN PSNR	Best classical (nb 01)	DnCNN gain over classical
75 (outside training)	12.11 dB	24.56 dB	not tuned	—

- **Against the paper**: CDnCNN-B reports about 33.9 / 31.2 / 27.9 dB on CBSD68 (68 of these 100 images). The model is within about 0.2 dB, despite fewer layers (17 vs 20), clipped noise and shorter training.
- **Outside the training range**: at σ = 65 and 75 it still gains 12–13 dB, so it generalises somewhat past σ = 55. But the gain stops growing: it rises from +8.9 dB at σ = 15 to +13.1 dB at σ = 65, then falls to +12.5 dB at σ = 75. Look at the σ = 65 crop to see what is left behind.
- **Loss plot**: falls quickly in the first few epochs, then flattens. Check on the plot whether it steps down at epochs 30 and 42, where the learning rate is cut.

Section 5 · Stretch (not done)

Swapping the on-the-fly Gaussian noise for random-quality JPEG would train a JPEG-artifact remover with the same architecture. This is the experiment notebook 05's JPEG result suggests: the training data defines what "restoration" means.

Check yourself

- [ ] What does "blind" mean, and why is it a fairer test than the tuned filters?
- [ ] Why predict the residual rather than the clean image?
- [ ] Why compute the target noise after clipping?
- [ ] Why augment on the fly instead of storing augmented patches?
- [ ] What does BatchNorm do, and why must you call `model.eval()` before testing?
- [ ] What does `GradScaler` protect against in mixed precision?
- [ ] Why does the learning rate drop at epochs 30 and 42?
- [ ] What is the predict-zero baseline for, and what does $10 \cdot \log_{10}(1 / \text{loss})$ mean?

05 · Does restoration help detection?

Purpose

Notebook 05 is where the research question gets answered. It asks: **when images are noisy, compressed or dark, how much detection accuracy is lost, how much does each restorer win back, and does the best-PSNR restorer also help detection most?** Short answer: restoration helps a lot on noise (DnCNN wins back 70–83% of the lost mAP), somewhat on JPEG, and in low light PSNR picks the worst method.

It takes `degradations.py` and `best_classical.json` from 01 and `dncnn_last.pt` from 04. It writes `detection_sweep.csv`, `recovered_percent.csv`, `per_image_ap.csv` (used by 06), `subset_ids.json` and the figures.

Design in one line: 300 COCO val2017 images × 3 degradations × 4 severities × 3–4 restorers, each scored by YOLOv8n with the official COCO mAP. That is 40 runs in total.

Setup

- **`find_input(name)`**: searches `/kaggle/input` recursively for a file. Kaggle's mounted folder names don't always match what the UI shows (`dcnn_last` vs `dcnn-last`), so searching is safer than typing paths.
- **Imports `degrade` from notebook 01's `degradations.py`** via `sys.path`, so the damage is identical to notebook 01's.
- **`resolve_checkpoint()`**: finds the `.pt` file. A `.pt` file is secretly a zip archive, and Kaggle unzips zips uploaded as datasets. So the checkpoint can arrive as a folder containing `data.pkl`. If that happens, this function zips the folder back into a working `.pt`, uncompressed and under an `archive/` prefix as torch expects. This fixed a real bug from the first run.
- Asserts a GPU is on, because DnCNN on 300 images × 12 conditions on a CPU would take hours.

Section 1 · COCO subset

- Downloads val2017 images and annotations if they're missing, checking each separately because an interrupted run can leave one without the other.
- **Why val, not train**: YOLOv8n was trained on COCO train. Testing on training images would overstate its accuracy.

- **Subset**: keeps images with at least one annotated object, shuffles with `random.Random(SEED)`, takes 300, and saves the ids to `subset_ids.json` so the subset is fixed and reportable. 2,035 objects in total.

Section 2 · Detector → COCO predictions

- **The class-id trap**: YOLO outputs class indices 0–79, but COCO category ids run 1–90 with gaps (there is no id 12, for example). YOLO's classes are COCO's 80 categories in increasing id order, so index i maps to the i-th smallest id: `cat_ids = sorted(coco_gt.getCatIds())`. The cell checks that all 80 names line up. Without this mapping, mAP comes out near zero for no visible reason.
- **`to_uint8_bgr`**: YOLO reads numpy arrays as BGR uint8, so the RGB float image is converted.
- **`detect(img, image_id)`**: runs YOLO with `conf=0.001`, `iou=0.7`, `max_det=300`, and converts boxes from (x1, y1, x2, y2) to COCO's (x, y, width, height). The confidence threshold is deliberately tiny: mAP averages over the whole precision–recall curve, so a high threshold would cut that curve short and lower the score unfairly.
- **`coco_map(results, ids)`**: runs `COCOeval` on exactly these image ids and returns mAP@[.5:.95] (averaged over box-overlap thresholds 0.5 to 0.95) and mAP@.5. Output is silenced except when asked.
- **`per_image_ap`**: the same evaluation one image at a time, for notebook 06's failure gallery.

Section 3 · Clean baseline

`load_coco_rgb` loads an image as float RGB. Clean mAP@[.5:.95] = **0.387**, close to YOLOv8n's published 0.373 on all 5,000 val images. The assert (mAP > 0.2) catches a broken setup before any degradation is applied.

Section 4 · Restorers

- **`DnCNN`**: the class is copied exactly from notebook 04, because the checkpoint stores only weights. Depth, channels and features are read from the checkpoint's saved config, so they can't mismatch.
- **`dncnn_restore`**: runs in full float32, with no autocast. Under float16 some activations overflowed to NaN on JPEG inputs, which are outside DnCNN's training data. The `isfinite` assert guards against this.

- **Classical filters**: the same four wrappers as notebook 01. `classical_restore(img, name, severity)` looks up the filter and parameters notebook 01 tuned for exactly this condition.
- `clahe_L`: CLAHE on Lab's L channel, the best contrast method from notebook 01's Experiment 3. Used only for low light.
- `CONDITIONS`: Gaussian σ 15, 25, 35, 50; JPEG q 70, 50, 30, 10; low light 0.4, 0.6, 0.8, 0.9. The asserts check that each one has a tuned classical entry.
- `RESTORERS`, `APPLIES`: which restorers run on which degradation.
- DnCNN on JPEG and low light is **deliberately out of distribution**: it never saw those in training. Whatever it does there is a finding, not a bug.

Section 5 · The main sweep

- **Resume**: rows already in `detection_sweep.csv` are skipped, and the CSV is saved after every condition, so a disconnect doesn't lose the run.
- **Paired inputs**: for each condition, all 300 degraded images are made **once**, with seeds from (image, degradation, severity). Every restorer then gets the identical degraded image.
- **For each restorer**: restore, clip, compute PSNR against the clean image, run detection, then compute mAP over all 300 images. Each row stores mAP, mAP@.5 and mean PSNR.
- **Per-image AP** is saved for four conditions notebook 06 needs (Gaussian 25 and 50, JPEG 10, low light 0.8), plus the clean images.
- Clean images are cached in RAM (about 1 GB) so they aren't re-read 36 times.

Section 6 · Figures and analysis

- **Figure 1, mAP vs severity**: one panel per degradation, one line per restorer, clean mAP as a dashed line.
- **Recovered share**:

```
\text{recovered} = \frac{\text{mAP}_{\text{restored}} -
\text{mAP}_{\text{degraded}}}{\text{mAP}_{\text{clean}} -
\text{mAP}_{\text{degraded}}}
```

100% means back to clean, 0% means no help, and negative means restoration made detection worse. This makes conditions with different amounts of damage comparable.

- **Figure 3, PSNR vs mAP scatter**, plus a loop that finds each condition where the best-PSNR restorer isn't the best-mAP restorer, and the Spearman rank correlation across all runs.

What the output tells you

Hypothesis	Outcome	Evidence
H1: mAP falls as damage grows	Confirmed	0.387 clean → 0.088 (Gaussian σ 50), 0.125 (JPEG q 10), 0.056 (low light 0.9)
H2: both help on noise, DnCNN more	Confirmed	DnCNN recovers 70–83% at every σ; classical falls from 60% to 27%. At σ 50: 0.304 vs 0.169
H3: JPEG barely recoverable, DnCNN may hurt	Half confirmed	DnCNN: −5% to +23%, and 1–2 dB *lower* PSNR. But bilateral recovers 41–48% at q 10–50 by smoothing block edges
H4: CLAHE best in low light	Rejected	CLAHE is worse than nothing (−4% to −21%): it amplifies the noise with the brightness. DnCNN is best (31–45%)
H5: best PSNR ≠ best mAP somewhere	Confirmed, for low light	In all 4 low-light conditions CLAHE has the best PSNR and the worst mAP

How to read the Spearman correlation (0.82): it looks like PSNR predicts mAP well, but both fall together as severity rises. The real question is the *ranking within one condition*. There, PSNR and mAP order the restorers the same way in 21 of 22 pairs on Gaussian noise and JPEG (the exception is JPEG q = 70, where almost nothing was lost), but reverse 17 of 24 restorer pairs in low light. This is the project's central finding: **PSNR is a reliable guide when the damage is noise or compression, and misleading when it is darkness.**

Check yourself

- [ ] Why use COCO val rather than train?
- [ ] What goes wrong without the class-id mapping, and why does index i map to the i-th smallest id?
- [ ] Why use `conf=0.001` instead of a normal threshold like 0.25?
- [ ] What does mAP@[.5:.95] average over?
- [ ] Why make the degraded images once per condition, before looping over restorers?
- [ ] Why can't the recovered share be computed from raw mAP differences alone?

- [ ] Why is a high Spearman correlation across all runs not enough to trust PSNR?
- [ ] Why does CLAHE raise PSNR but lower mAP in low light?

06 · Opening up the denoiser

Purpose

Notebook 06 is the interpretability part. It opens up the trained DnCNN to explain notebook 05's numbers. It answers: **what did the network learn, and why does it win on noise but fail on JPEG and blur?** Five probes: first-layer filters, receptive field, frequency response, the predicted residual, and a failure gallery.

Inputs: the checkpoint from 04, `per_image_ap.csv` and `subset_ids.json` from 05, `degradations.py` and `best_classical.json` from 01, and the BSD500 val images.

Setup

- **`find_input`**, **`resolve_checkpoint`**, **`DnCNN`**: the same as notebook 05. The model is rebuilt from the checkpoint's saved config and put in `eval()` mode.
- **`residual(img)`**: returns what the network thinks the noise is. **`denoise(img)`**: input minus residual, clipped.
- **`noisy_like_nb01`**: the same noise as notebooks 01 and 04 for test image i, so every result here lines up with earlier ones.
- `test_imgs` is the same 96 test images.

Section 1 · First-layer filters

- **`w1`**: the first layer's weights, shape (64, 3, 3, 3): 64 filters, each 3 × 3 over R, G and B. The plot shows each as a tiny RGB image, normalised to its own range.
- **`KERNELS`**: textbook 3 × 3 kernels to compare against: Gaussian (smoothing), Laplacian (edges in all directions), Sobel x and y (vertical and horizontal edges), identity.
- **`match_filters(w)`**: for each filter, averages over colour and finds the textbook kernel with the highest |cosine similarity|. The absolute value is used because a negated Sobel is still an edge detector. It also computes **`colour_share`**: the fraction of the filter's energy in differences *between* channels. Near 0 means the filter treats R, G and B alike; near 1 means it compares colours (e.g. red minus green).
- **Result**: only 9 of 64 filters closely match a textbook kernel (|cosine| ≥ 0.8): 6 Sobel (3 x, 3 y), 1 Laplacian, 1 Gaussian, 1 identity. The median colour share is 0.79, and 43 of 64 filters are mostly colour-opponent. Nobody told the network about colour spaces, yet it learned to separate brightness from colour, the same structure notebook 01's colour-space experiment found by hand.

- **Caveat**: a 3 × 3 filter has only 9 numbers per channel, so "looks like Sobel" is a loose claim. The solid finding is the split into smoothing, edge and colour-difference filters.

Section 2 · Receptive field

- `rf_theory(depth, k)` = 1 + depth · (k − 1). Each stride-1 3 × 3 layer widens the view by 2 pixels, so 17 layers see 35 × 35 (±17 px). The assert checks this.
- `effective_rf(img, sigma)`: measures which inputs actually matter. It runs a noisy crop through the model with `requires_grad`, back-propagates from the **centre output pixel** only, and takes |∂ output / ∂ input| summed over channels. This is a heatmap of how much each input pixel affects that one output.
- `mass_radius(g, frac)`: the half-width of the smallest centred square holding 90% of that gradient.
- **Result**: 90% of the influence comes from within ±7 px at σ = 15 and 25, widening to ±10 px at σ = 50. Nothing outside the 35 × 35 box has any influence, as the architecture requires (a free correctness check). So the network mostly looks nearby, and looks further when the noise is heavier and it needs more evidence to average.

Section 3 · Frequency response

- `radial_power_spectrum(gray)`: FFT of the image, power $|F|^2$, averaged over rings of equal frequency. This shows how much of the image is at each level of detail, from coarse (low frequency) to fine (high frequency).
- `luma(img)`: converts to brightness, so one spectrum per image.
- `classical_gauss(img, sigma)`: notebook 01's tuned classical filter for that σ.
- `transfer(denoise_fn, sigma)`: P_out(f) / P_noisy(f), averaged over 10 images. This is the denoiser's empirical transfer function: 1 means a frequency passes untouched, near 0 means it's removed.
- `cutoff(f, ratio)`: the lowest frequency where output power drops below half of the input's.
- **Why it matters**: white noise is flat across frequencies, while natural images fall off roughly as $1/f^2$. So at high frequency the image is mostly noise, and any denoiser has to cut there.
- **Result**: both methods move their cutoff lower as the noise grows. But DnCNN's cutoff is up to 1.9× higher (at σ = 25), and at σ = 50 it keeps about 6× more of the highest frequencies than the Gaussian filter. Those frequencies carry edges and texture, which is the likeliest reason DnCNN wins back so much more mAP on noise. It is not a fixed low-pass filter; it adapts to the noise level.

Section 4 · The predicted residual

- `edge_strength(clean)`: Sobel gradient magnitude of the clean image, i.e. where its edges are.
- `structure_leak(clean, res)`: correlation between |predicted residual| and edge strength. About 0 means the residual looks like noise. Higher means the network is removing real image structure.
- `PROBES`: feeds in one in-distribution input (Gaussian) and four it never trained on.

Input	PSNR gain	Leak	Residual std (0–255)
Gaussian σ 25	+10.56 dB	−0.02	22.7
Salt & pepper 0.05	+0.24 dB	−0.01	9.6
JPEG q 10	−1.08 dB	0.05	51.7
Blur 2.0	−1.73 dB	0.05	156.9
Low light 0.8	−0.10 dB	0.05	11.8

How to read it: on Gaussian noise the residual is structureless and close to the true noise level (std 22.7 for σ = 25). On JPEG and blur the network still "finds noise", but its residual is huge (51.7 and 156.9) and traces edges and block boundaries. It is adding streak artifacts, so PSNR drops. In low light the residual looks like noise, and the output is cleaner but just as dark. That explains notebook 05's H5 reversal: DnCNN removes the noise the detector cares about but not the darkness PSNR cares about, while CLAHE does the opposite.

The next cell checks that on Gaussian input the predicted residual matches the true noise's size and correlates with it.

Section 5 · Failure gallery

- Loads per-image AP from notebook 05 for Gaussian σ = 50 and pivots it to one row per image.
- **The "paradox" set**: images where DnCNN had *higher* PSNR than the classical filter but *lower* AP. The cell prints how many there are, then picks the 3 worst plus the 1 image where DnCNN helped most.
- **Recreating the images**: it uses exactly notebook 05's seeds, so the degraded images are identical to the ones that were scored.

- **`load_coco`**: downloads just the needed image if it isn't attached. **`draw`**: runs YOLO at `conf=0.25` (a readable number of boxes for a figure, unlike notebook 05's 0.001 for scoring) and draws the labelled boxes.
- **What the gallery shows** (now written up in the notebook's Findings): in three images, DnCNN smoothed away fine texture (the cat's fur and wooden chair in #353970, the bird's feathers in #315257, the wicker chairs in #210099), and YOLO lost or split the objects even though PSNR went up. In #545129 it's the other way round: the classical Gaussian blur washed out the zebras' stripes (YOLO saw "dog"s and a "couch", AP 0.00), while DnCNN kept them and found all three zebras (AP 0.96). This matches DnCNN's higher frequency cutoff in section 3.

Check yourself

- [ ] Why use the absolute cosine similarity when matching filters?
- [ ] What does a high colour share mean, and how does it connect to notebook 01's colour-space result?
- [ ] Derive the receptive field for depth 17. Why is the effective one so much smaller?
- [ ] How does back-propagating from one output pixel measure what the network "looks at"?
- [ ] What is an empirical transfer function, and what does DnCNN's higher cutoff mean?
- [ ] What does the structure leak measure, and why is it near 0 for Gaussian input?
- [ ] Why does DnCNN lower PSNR on blur and JPEG?
- [ ] Use sections 3 and 4 to explain both halves of notebook 05's low-light result.